CEN206 Object-Oriented Programming

Design Patterns Introduction & Creational Patterns

Author: Asst. Prof. Dr. Ugur CORUH

List of Figures

1 center . . . 5
2 center . . . 8
3 center . . . 10
4 center . . . 15
5 center . . . 18
6 center . . . 24
7 center . . . 27
8 center . . . 33
9 center . . . 36
10 center . . . 42
11 center . . . 43

List of Tables

CEN206 Object-Oriented Programming

Week-9 (Design Patterns Introduction & Creational Patterns)

Spring Semester, 2025-2026 Download DOC-PDF[1], DOC-DOCX[2], SLIDE[3]

Week-9 Outline

Modules

- **Module A:** What Are Design Patterns?
- **Module B:** Factory Method Pattern
- **Module C:** Abstract Factory Pattern
- **Module D:** Builder Pattern
- **Module E:** Prototype Pattern
- **Module F:** Singleton Pattern

Each module includes comprehensive coverage of intent, problem, solution, structure, pseudocode, applicability, implementation steps, pros/cons, relationships with other patterns, and Java code examples.

Module A: What Are Design Patterns?

---

[1] ce204-week-9.en.md_doc.pdf
[2] ce204-week-9.en.md_word.docx
[3] ce204-week-9.en.md_slide.pdf

What Are Design Patterns?

- **Design patterns** are typical solutions to commonly occurring problems in software design.
- They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.
- A pattern is not a specific piece of code, but a general concept for solving a particular problem.
- You can follow the pattern details and implement a solution that suits the realities of your own program.
- Patterns are often confused with algorithms. While an algorithm always defines a clear set of actions, a pattern is a more high-level description of a solution.
- The code of the same pattern applied to two different programs may be different.

Reference: RefactoringGuru - Design Patterns[4]

Patterns vs. Algorithms

Aspect	Algorithm	Design Pattern
Level	Low-level, step-by-step	High-level, architectural
Analogy	A cooking recipe	A blueprint
Specificity	Clear set of actions to achieve a goal	General concept for solving a problem
Implementation	Same code can be reused directly	Must be adapted to each program
Focus	How to process data	How to structure code

- An algorithm is like a cooking recipe: both have clear steps to achieve a goal.
- A pattern is more like a blueprint: you can see what the result and its features are, but the exact order of implementation is up to you.

What Does a Pattern Consist Of?

Most patterns are described formally so people can reproduce them in many contexts. A pattern description usually includes:

- **Intent**: briefly describes both the problem and the solution.
- **Motivation**: further explains the problem and the solution the pattern makes possible.
- **Structure**: shows each part of the pattern and how they are related (class diagrams).
- **Pseudocode/Code example**: makes it easier to grasp the idea behind the pattern.
- **Applicability**: when to use the pattern.
- **Implementation steps**: how to implement the pattern.
- **Relations with other patterns**: how patterns relate to each other.

Some pattern catalogs also list: pros and cons, known uses, and alternative names.

History of Design Patterns

- The concept of patterns was first described by **Christopher Alexander** in the book *A Pattern Language: Towns, Buildings, Construction* (1977).
  - The book describes a "language" for designing the urban environment.
- The idea was picked up by four authors: **Erich Gamma**, **John Vlissides**, **Ralph Johnson**, and **Richard Helm**.
- In 1994, they published **"Design Patterns: Elements of Reusable Object-Oriented Software"**.

[4] https://refactoring.guru/design-patterns

- The book featured **23 patterns** solving various problems of object-oriented design.
- Due to the long name of the book, people started calling it **"the book by the Gang of Four"** (GoF).
- Since then, dozens of other object-oriented patterns have been discovered.

The Gang of Four (GoF)

The four authors of the seminal book:

Author	Contribution
Erich Gamma	Later led Eclipse platform development at IBM
Richard Helm	Software architect in Australia
Ralph Johnson	Professor at University of Illinois
John Vlissides	Researcher at IBM (1961-2005)

Their book cataloged 23 classic design patterns that remain foundational to OOP today. The patterns are solutions that evolved organically through real-world software development experience.

Classification of Design Patterns

Design patterns differ by their complexity, level of detail, and scale of applicability. The GoF classified them into **three categories**:

Category	Count	Purpose
Creational	5	Object creation mechanisms
Structural	7	Assembling objects and classes into larger structures
Behavioral	10	Communication between objects
Total	**22**	(23 in GoF, but we cover 22 core patterns)

Patterns can also be classified by their scope: - **Class patterns**: deal with relationships between classes and subclasses (compile-time) - **Object patterns**: deal with object relationships (runtime)

GoF Pattern Categories - Overview

Creational Patterns (5)

Creational patterns provide object creation mechanisms that increase flexibility and reuse of existing code.

Pattern	Purpose
Factory Method	Creates objects via an interface; subclasses decide the type
Abstract Factory	Produces families of related objects without specifying concrete classes
Builder	Constructs complex objects step by step
Prototype	Copies existing objects without depending on their classes

Pattern	Purpose
Singleton	Ensures a class has only one instance with a global access point

---

Structural Patterns (7)

Structural patterns explain how to assemble objects and classes into larger structures while keeping them flexible and efficient.

Pattern	Purpose
Adapter	Allows objects with incompatible interfaces to collaborate
Bridge	Splits a large class into two hierarchies (abstraction + implementation)
Composite	Composes objects into tree structures
Decorator	Attaches new behaviors to objects by wrapping them
Facade	Provides a simplified interface to a library or framework
Flyweight	Fits more objects into RAM by sharing common state
Proxy	Provides a substitute or placeholder for another object

---

Behavioral Patterns (10)

Behavioral patterns take care of effective communication and the assignment of responsibilities between objects.

Pattern	Purpose
Chain of Responsibility	Passes requests along a chain of handlers
Command	Turns a request into a stand-alone object
Iterator	Traverses elements of a collection sequentially
Mediator	Reduces chaotic dependencies between objects
Memento	Saves and restores the previous state of an object
Observer	Defines a subscription mechanism for event notification
State	Alters behavior when internal state changes
Strategy	Defines a family of interchangeable algorithms
Template Method	Defines the skeleton of an algorithm; subclasses fill in steps
Visitor	Separates algorithms from the objects on which they operate

---

Why Learn Design Patterns?

- **Proven solutions**: Patterns are tried-and-tested solutions to common problems. Even if you never encounter these problems, knowing patterns teaches you how to solve problems using OOP principles.

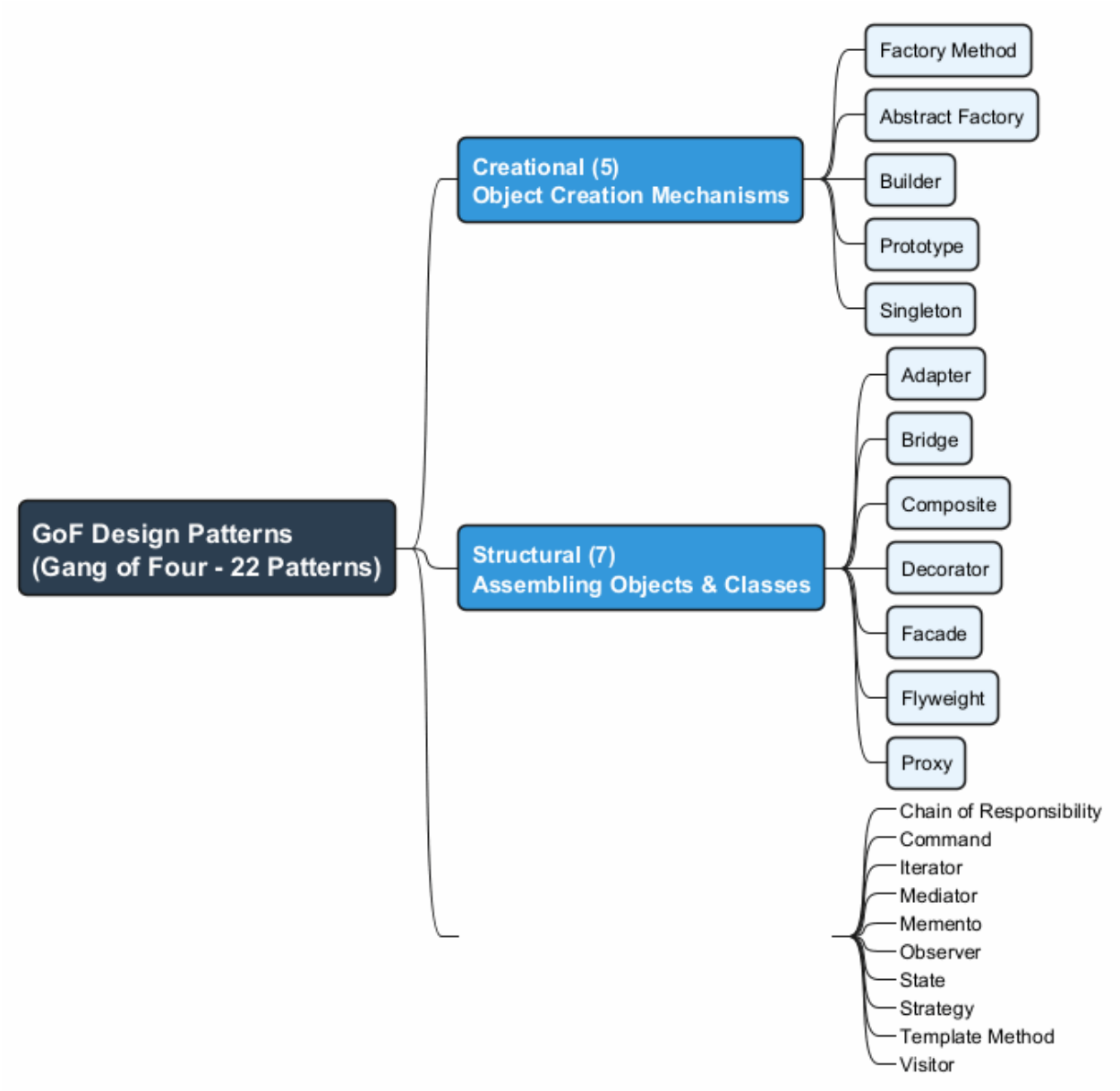


Figure 1: center

- **Common language**: Patterns define a shared vocabulary. You can say "use a Singleton for that" and everyone immediately understands the suggestion.
- **Better communication**: When team members know patterns, communication about design is faster and more precise.
- **Understand frameworks**: Many frameworks and libraries use design patterns. Understanding patterns helps you comprehend how frameworks work.
- **Write better code**: Patterns help you write code that is more flexible, reusable, and maintainable.
- **Career growth**: Design pattern knowledge is frequently tested in technical interviews.

Benefits of Design Patterns

- **Reusability**: Patterns provide reusable solutions that can be adapted across projects.
- **Maintainability**: Code structured with patterns is easier to understand and modify.
- **Scalability**: Patterns help design systems that can grow without becoming unmanageable.
- **Flexibility**: Patterns often make it easier to swap implementations or add new features.
- **Documentation**: Patterns serve as a form of documentation for design decisions.

Criticism of Design Patterns

- **Over-engineering**: Using patterns where simple code would suffice adds unnecessary complexity.
- **Language limitations**: Patterns often compensate for missing language features (e.g., Strategy in languages without first-class functions).
- **Misuse**: Applying the wrong pattern to a problem can make code worse, not better.
- **Learning curve**: Understanding when and how to apply patterns requires experience.

Module A: Takeaways

- Design patterns are **typical solutions** to commonly occurring problems in software design.
- They were popularized by the **Gang of Four (GoF)** book in 1994 with **23 patterns**.
- Patterns are classified into three categories: **Creational** (5), **Structural** (7), and **Behavioral** (10).
- Each pattern has: **Intent**, **Motivation**, **Structure**, **Pseudocode**, **Applicability**, **Implementation**, and **Relations**.
- Learning patterns gives you a **common vocabulary** and helps you write **better, more maintainable code**.
- Patterns should be applied thoughtfully; **over-engineering** and **misuse** are real risks.
- In this week, we focus on the **5 Creational Patterns**: Factory Method, Abstract Factory, Builder, Prototype, and Singleton.

Module B: Factory Method Pattern

Factory Method Pattern - Overview

- **Also Known As**: Virtual Constructor
- **Category**: Creational Pattern
- **Complexity**: Low
- **Popularity**: High

Intent: Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

Reference: RefactoringGuru - Factory Method[5]

[5] https://refactoring.guru/design-patterns/factory-method

Factory Method - The Problem

Imagine that you are creating a logistics management application. The first version of your app can only handle transportation by trucks, so the bulk of your code lives inside the `Truck` class.

After a while, your app becomes pretty popular. Each day you receive dozens of requests from sea transportation companies to incorporate sea logistics into the app.

What happens if you add `Ship` directly?

- Adding a new class isn't that simple if the rest of the code is already coupled to existing classes.
- Most of your code is coupled to the `Truck` class.
- Adding `Ship` would require making changes to the entire codebase.
- Adding another transportation type later would require all of these changes again.
- The result: pretty nasty code, riddled with conditionals that switch behavior depending on the class of transportation objects.

Factory Method - The Solution

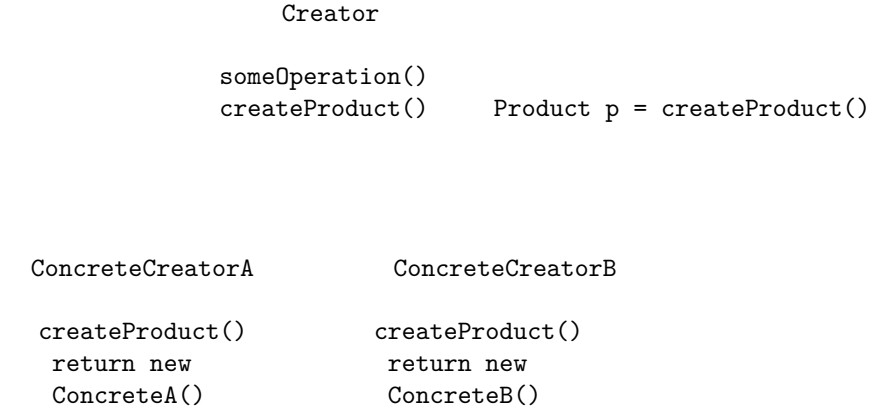
The Factory Method pattern suggests that you replace direct object construction calls (using the `new` operator) with calls to a special **factory method**.

- Objects are still created via the `new` operator, but it is called from within the factory method.
- Objects returned by a factory method are often referred to as **products**.

Key insight: Subclasses can override the factory method to change the class of products being created.

- There is a slight limitation: subclasses may return different types of products only if these products have a **common base class or interface**.
- Also, the factory method in the base class should have its return type declared as this interface.

Factory Method - Solution Diagram



- The `Creator` class has a factory method `createProduct()`.
- Each `ConcreteCreator` overrides the factory method to return a different product type.
- All products implement the same interface.

Factory Method - Structure

The Factory Method pattern has four key participants:

1. **Product** (Interface/Abstract Class)
   - Declares the interface common to all objects that the factory method can produce.
2. **Concrete Products**
   - Different implementations of the product interface.
3. **Creator** (Abstract Class)
   - Declares the factory method that returns new product objects. Its return type must match the product interface.
   - May contain some core business logic that relies on product objects.
4. **Concrete Creators**
   - Override the base factory method to return a different type of product.
   - The factory method does not have to create new instances all the time; it can also return existing objects from a cache, object pool, or other source.

Factory Method - Class Diagram

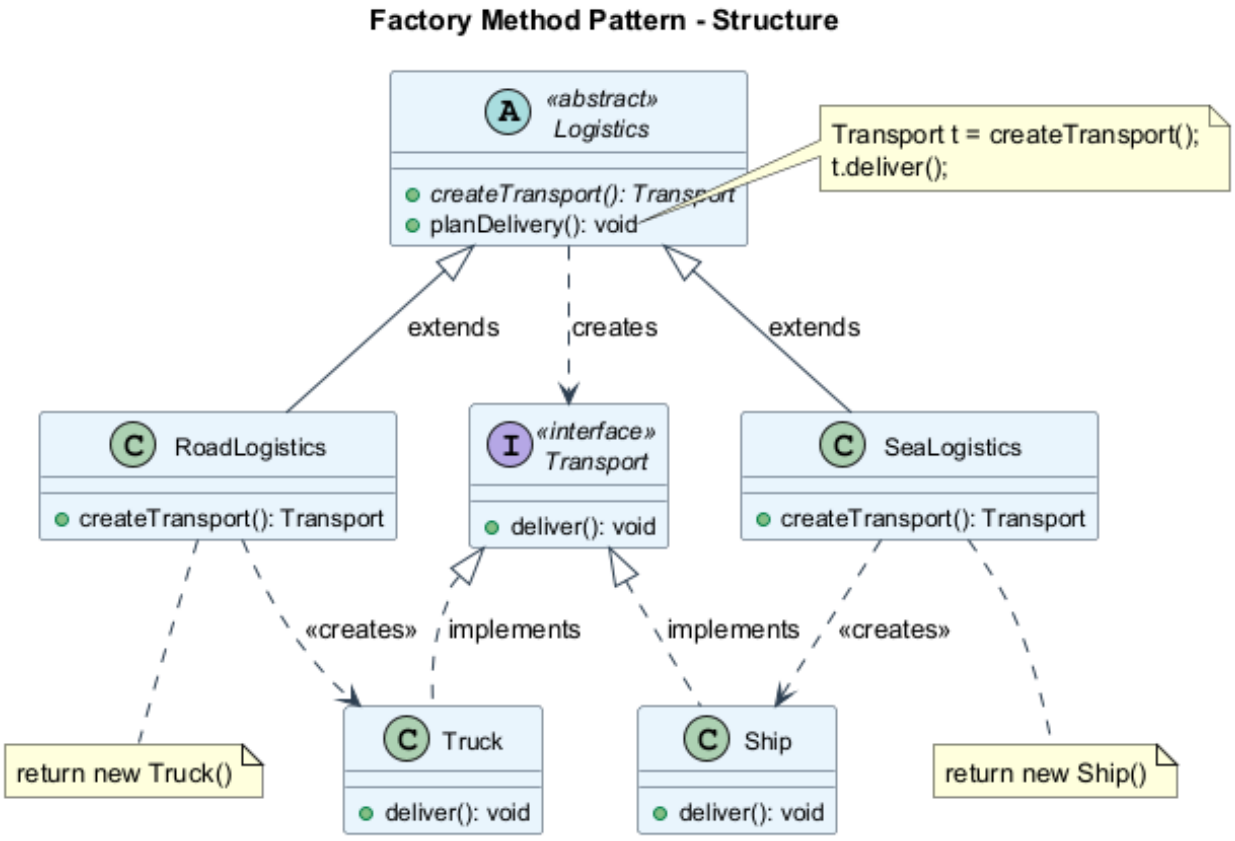


Figure 2: center

Factory Method - Pseudocode

This example illustrates how the Factory Method can be used for creating cross-platform UI elements without coupling the client code to concrete UI classes.

```
// The creator class declares the factory method
// that must return an object of a product class.
class Dialog is
    abstract method createButton(): Button

    method render() is
        Button okButton = createButton()
        okButton.onClick(closeDialog)
        okButton.render()

// Concrete creators override the factory method
class WindowsDialog extends Dialog is
    method createButton(): Button is
        return new WindowsButton()

class WebDialog extends Dialog is
    method createButton(): Button is
        return new HTMLButton()
```

Factory Method - Pseudocode (cont.)

```
// The product interface declares the operations
// that all concrete products must implement.
interface Button is
    method render()
    method onClick(f)

// Concrete products provide various implementations.
class WindowsButton implements Button is
    method render(a, b) is
        // Render a button in Windows style.
    method onClick(f) is
        // Bind a native OS click event.

class HTMLButton implements Button is
    method render(a, b) is
        // Return an HTML representation of a button.
    method onClick(f) is
        // Bind a web browser click event.
```

Factory Method - Pseudocode (cont.)

```
class Application is
    field dialog: Dialog

    // The application picks a creator's type depending
    // on the current configuration or environment.
    method initialize() is
        config = readApplicationConfigFile()
        if (config.OS == "Windows") then
            dialog = new WindowsDialog()
        else if (config.OS == "Web") then
            dialog = new WebDialog()
        else
```

```
        throw new Exception("Error! Unknown OS.")

// The client code works with an instance of a
// concrete creator, albeit through its base interface.
method main() is
    this.initialize()
    dialog.render()
```

Factory Method - Sequence Diagram

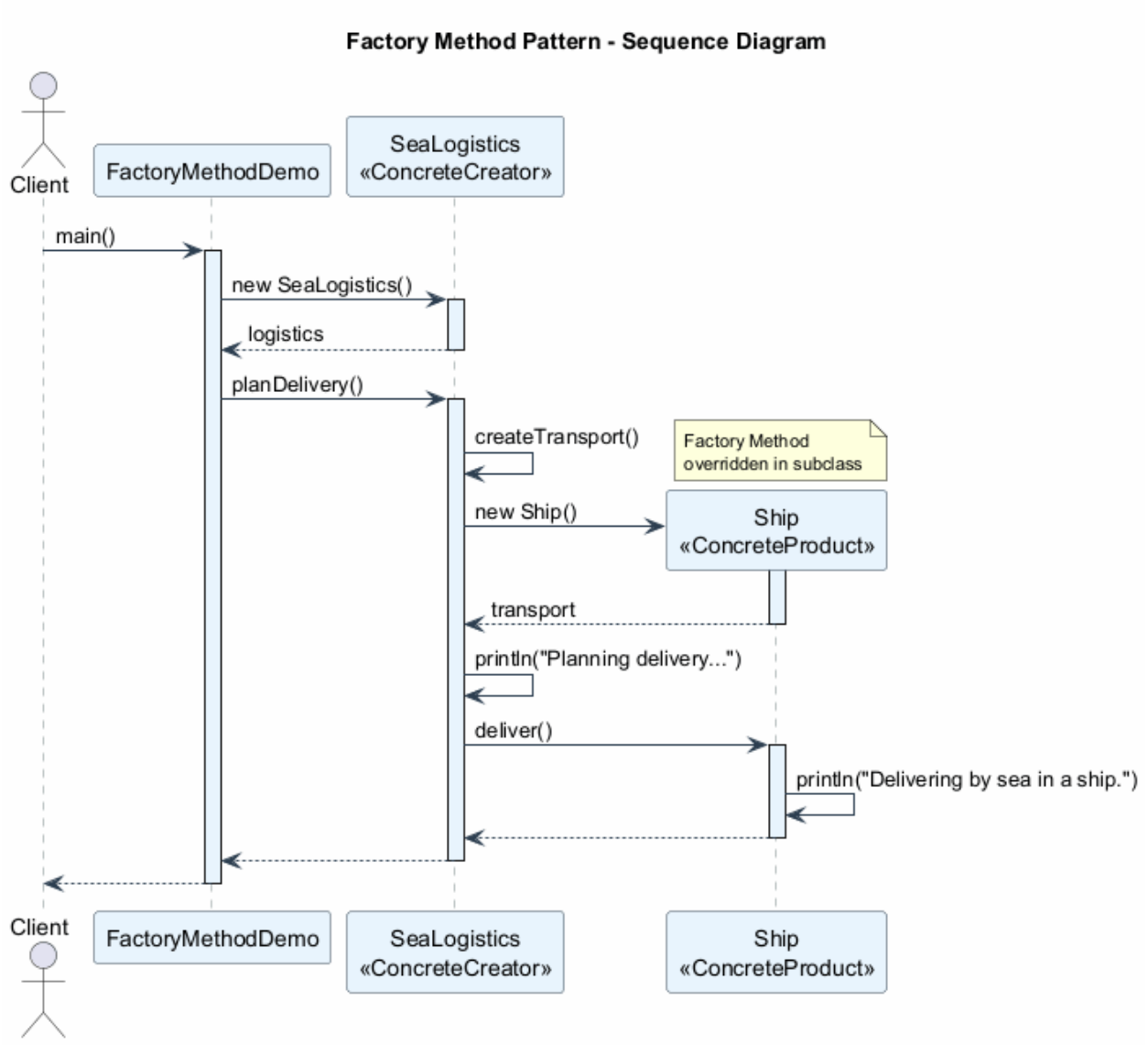


Figure 3: center

Factory Method - Applicability

Use the Factory Method pattern when:

1. **You don't know beforehand the exact types and dependencies of the objects** your code

should work with.
    - The Factory Method separates product construction code from the code that actually uses the product. Therefore it is easier to extend the product construction code independently from the rest of the code.
2. **You want to provide users of your library or framework with a way to extend its internal components.**
    - Users can subclass your framework classes and override factory methods to extend them with their own types.
3. **You want to save system resources by reusing existing objects** instead of rebuilding them each time.
    - You need a regular method capable of creating new objects as well as reusing existing ones. That sounds very much like a factory method.

Factory Method - How to Implement

1. **Make all products follow the same interface.** This interface should declare methods that make sense in every product.
2. **Add an empty factory method inside the creator class.** The return type of the method should match the common product interface.
3. **In the creator's code, find all references to product constructors.** One by one, replace them with calls to the factory method, while extracting the product creation code into the factory method.
4. **Create a set of concrete creator subclasses** for each type of product listed in the factory method. Override the factory method in the subclasses and extract the appropriate bits of construction code from the base method.
5. **If there are too many product types** and it does not make sense to create subclasses for all of them, you can reuse the control parameter from the base class in subclasses.
6. **If, after all of the extractions, the base factory method has become empty**, you can make it abstract. If there is something left, you can make it a default behavior of the method.

Factory Method - Pros and Cons

Pros

- **Avoids tight coupling** between the creator and the concrete products.
- **Single Responsibility Principle (SRP)**: You can move the product creation code into one place in the program, making the code easier to support.
- **Open/Closed Principle (OCP)**: You can introduce new types of products into the program without breaking existing client code.

Cons

- **Code complexity increases**: The code may become more complicated since you need to introduce many new subclasses to implement the pattern. The best case scenario is when you are introducing the pattern into an existing hierarchy of creator classes.

Factory Method - Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated, more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complex).
- **Abstract Factory** classes are often based on a set of **Factory Methods**, but you can also use **Prototype** to compose the methods on these classes.

- You can use **Factory Method** along with **Iterator** to let collection subclasses return different types of iterators that are compatible with the collections.
- **Prototype** is not based on inheritance, so it does not have its drawbacks. On the other hand, Prototype requires a complicated initialization of the cloned object. **Factory Method** is based on inheritance but does not require an initialization step.
- **Factory Method** is a specialization of **Template Method**. At the same time, a Factory Method may serve as a step in a large Template Method.

Factory Method - Java Code Example

```java
// Product interface
interface Transport {
    void deliver();
}

// Concrete Products
class Truck implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivering by land in a truck.");
    }
}

class Ship implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivering by sea in a ship.");
    }
}
```

Factory Method - Java Code Example (cont.)

```java
// Creator (abstract)
abstract class Logistics {
    // Factory Method
    public abstract Transport createTransport();

    // Business logic that uses the factory method
    public void planDelivery() {
        Transport transport = createTransport();
        System.out.println("Planning delivery...");
        transport.deliver();
    }
}

// Concrete Creators
class RoadLogistics extends Logistics {
    @Override
    public Transport createTransport() {
        return new Truck();
    }
}
```

```java
class SeaLogistics extends Logistics {
    @Override
    public Transport createTransport() {
        return new Ship();
    }
}
```

Factory Method - Java Code Example (cont.)

```java
// Client code
public class FactoryMethodDemo {
    public static void main(String[] args) {
        Logistics logistics;

        // Configuration determines which factory to use
        String transportType = "sea"; // could come from config

        if (transportType.equals("road")) {
            logistics = new RoadLogistics();
        } else if (transportType.equals("sea")) {
            logistics = new SeaLogistics();
        } else {
            throw new IllegalArgumentException(
                "Unknown transport type: " + transportType
            );
        }

        // Client works with the abstract creator
        logistics.planDelivery();
        // Output: Planning delivery...
        //         Delivering by sea in a ship.
    }
}
```

Module B: Takeaways

- **Factory Method** provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects created.
- It solves the problem of **tight coupling** between creator code and concrete product classes.
- The pattern uses **four participants**: Product interface, Concrete Products, Creator class, and Concrete Creators.
- Factory Method supports the **Open/Closed Principle** (new products without modifying existing code) and **Single Responsibility Principle** (centralized creation logic).
- It is often the **starting point** for more complex patterns like Abstract Factory or Prototype.
- The main trade-off is **increased number of subclasses** in the codebase.
- Factory Method is a specialization of the **Template Method** pattern.

Module C: Abstract Factory Pattern

Abstract Factory Pattern - Overview

- **Category**: Creational Pattern
- **Complexity**: Medium
- **Popularity**: High

Intent: Abstract Factory is a creational design pattern that lets you produce **families of related objects** without specifying their concrete classes.

Reference: RefactoringGuru - Abstract Factory[6]

Abstract Factory - The Problem

Imagine that you are creating a furniture shop simulator. Your code consists of classes that represent:

- A family of related products, say: `Chair` + `Sofa` + `CoffeeTable`.
- Several variants of this family. For example, products `Chair` + `Sofa` + `CoffeeTable` are available in these variants: **Modern**, **Victorian**, **ArtDeco**.

You need a way to create individual furniture objects so that they match other objects of the same family. Customers get quite mad when they receive non-matching furniture.

Problems with direct construction:

- You don't want to change existing code when adding new products or families of products.
- Furniture vendors update their catalogs very often, and you wouldn't want to change the core code each time it happens.

Abstract Factory - The Solution

The Abstract Factory pattern suggests:

1. **Explicitly declare interfaces** for each distinct product of the product family (e.g., `Chair`, `Sofa`, `CoffeeTable`).
2. **Make all variants** of products follow those interfaces. For example, all chair variants implement the `Chair` interface; all coffee tables implement the `CoffeeTable` interface, and so on.
3. **Declare the Abstract Factory** – an interface with a list of creation methods for all products that are part of the product family (e.g., `createChair`, `createSofa`, `createCoffeeTable`).
4. **Create separate factory classes** for each variant of the product family. A factory is a class that returns products of a particular kind. For example, the `ModernFurnitureFactory` can only create `ModernChair`, `ModernSofa`, and `ModernCoffeeTable` objects.

Abstract Factory - Solution Diagram

```
    AbstractFactory

createChair()
createSofa()
createCoffeeTable()
```

```
ModernFactory              VictorianFactory
```

[6] https://refactoring.guru/design-patterns/abstract-factory

```
createChair()            createChair()
  → ModernCh               → VictorCh
createSofa()             createSofa()
  → ModernSf               → VictorSf
```

The client code works with factories and products only through their abstract interfaces. This lets you change the type of factory (and thus the variant of products) without changing the client code.

Abstract Factory - Structure

The Abstract Factory pattern has five key participants:

1. **Abstract Products** declare interfaces for a set of distinct but related products which make up a product family.
2. **Concrete Products** are various implementations of abstract products, grouped by variants. Each abstract product (chair/sofa) must be implemented in all given variants (Victorian/Modern).
3. **Abstract Factory** interface declares a set of methods for creating each of the abstract products.
4. **Concrete Factories** implement creation methods of the abstract factory. Each concrete factory corresponds to a specific variant of products and creates only those product variants.
5. **Client** works with both factories and products through abstract interfaces. This lets the client work with any factory/product variant without coupling to concrete classes.

Abstract Factory - Class Diagram

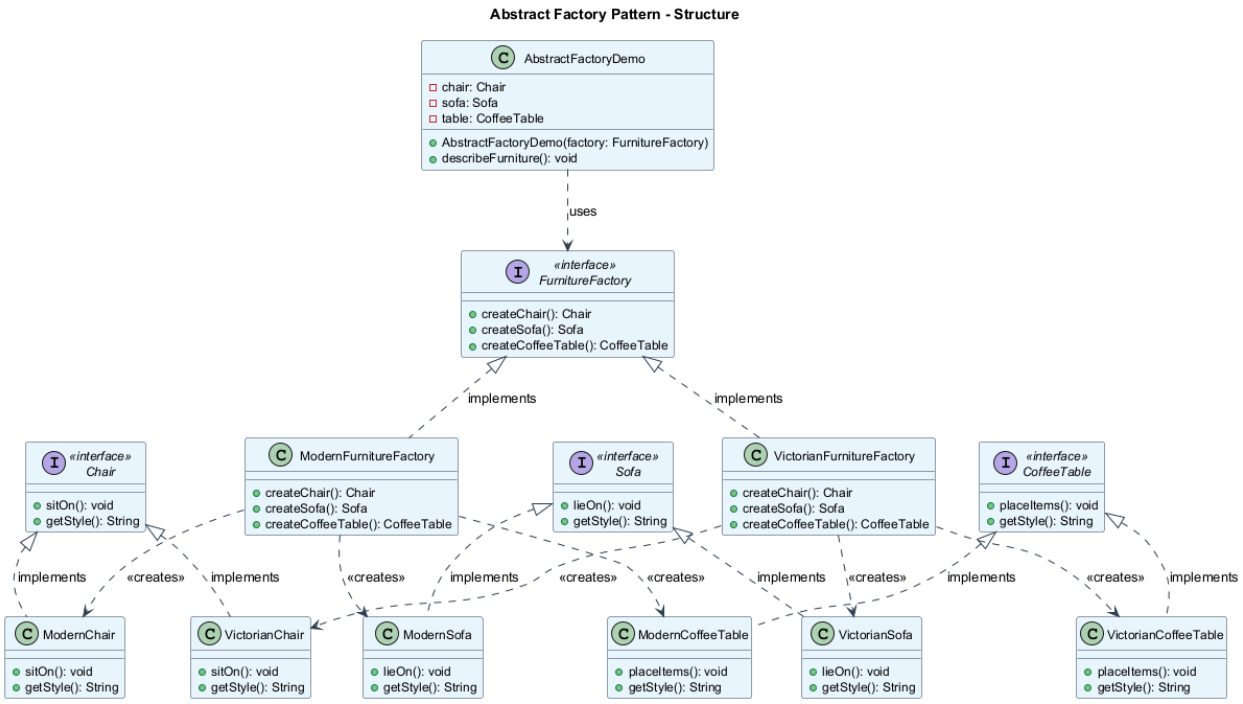


Figure 4: center

Abstract Factory - Pseudocode

This example illustrates how the Abstract Factory pattern can be used for creating cross-platform UI elements without coupling the client code to concrete UI classes.

```
// The abstract factory interface declares a set of methods
// that return different abstract products.
interface GUIFactory is
    method createButton(): Button
    method createCheckbox(): Checkbox

// Concrete factories produce a family of products that
// belong to a single variant.
class WinFactory implements GUIFactory is
    method createButton(): Button is
        return new WinButton()
    method createCheckbox(): Checkbox is
        return new WinCheckbox()

class MacFactory implements GUIFactory is
    method createButton(): Button is
        return new MacButton()
    method createCheckbox(): Checkbox is
        return new MacCheckbox()
```

Abstract Factory - Pseudocode (cont.)

```
// Each distinct product of a product family should
// have a base interface.
interface Button is
    method paint()

// Each concrete product is created in a corresponding
// concrete factory.
class WinButton implements Button is
    method paint() is
        // Render a button in Windows style.

class MacButton implements Button is
    method paint() is
        // Render a button in macOS style.

interface Checkbox is
    method paint()

class WinCheckbox implements Checkbox is
    method paint() is
        // Render a checkbox in Windows style.

class MacCheckbox implements Checkbox is
    method paint() is
        // Render a checkbox in macOS style.
```

Abstract Factory - Pseudocode (cont.)

```
// The client code works with factories and products
// only through abstract types: GUIFactory, Button, Checkbox.
class Application is
    private field factory: GUIFactory
    private field button: Button

    constructor Application(factory: GUIFactory) is
        this.factory = factory

    method createUI() is
        this.button = factory.createButton()

    method paint() is
        button.paint()

// The application picks the factory type depending on
// the current configuration or environment settings.
class ApplicationConfigurator is
    method main() is
        config = readApplicationConfigFile()
        if (config.OS == "Windows") then
            factory = new WinFactory()
        else if (config.OS == "Mac") then
            factory = new MacFactory()
        else
            throw new Exception("Error! Unknown OS.")
        Application app = new Application(factory)
```

Abstract Factory - Sequence Diagram

Abstract Factory - Applicability

Use the Abstract Factory pattern when:

1. **Your code needs to work with various families of related products**, but you don't want it to depend on the concrete classes of those products – they might be unknown beforehand or you simply want to allow for future extensibility.
   - The Abstract Factory provides you with an interface for creating objects from each class of the product family. As long as your code creates objects via this interface, you don't have to worry about creating the wrong variant of a product which doesn't match the products already created by your app.
2. **You have a class with a set of Factory Methods** that blur its primary responsibility.
   - In a well-designed program, each class is responsible only for one thing (SRP). When a class deals with multiple product types, it may be worth extracting its factory methods into a stand-alone factory class or a full-blown Abstract Factory implementation.

Abstract Factory - How to Implement

1. **Map out a matrix** of distinct product types versus variants of these products.
2. **Declare abstract product interfaces** for all product types. Then make all concrete product classes implement these interfaces.

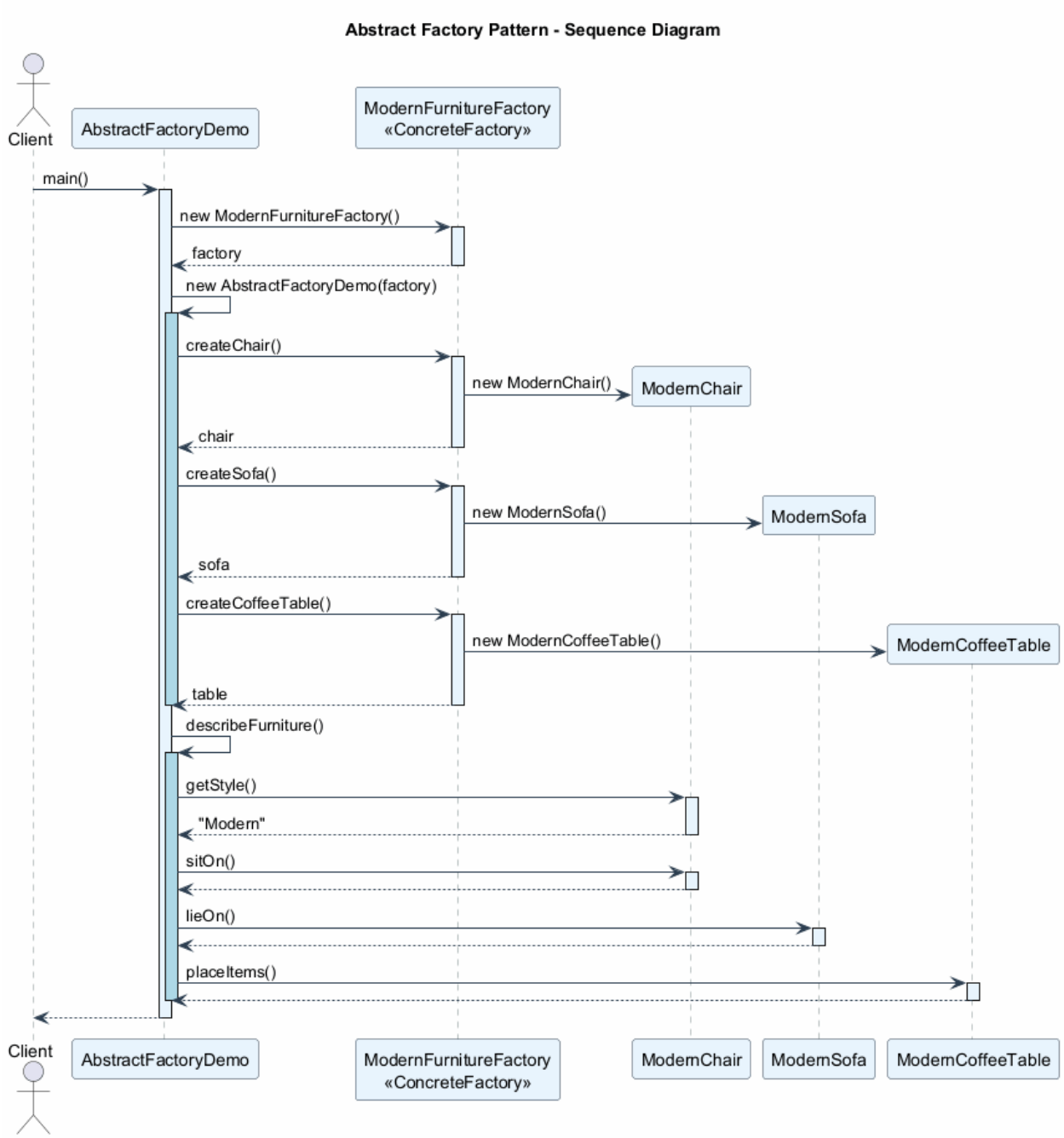


Figure 5: center

3. **Declare the abstract factory interface** with a set of creation methods for all abstract products.
4. **Implement a set of concrete factory classes**, one for each product variant.
5. **Create factory initialization code** somewhere in the app. It should instantiate one of the concrete factory classes, depending on the application configuration or the current environment. Pass this factory object to all classes that construct products.
6. **Scan through the code** and find all direct calls to product constructors. Replace them with calls to the appropriate creation method on the factory object.

Abstract Factory - Pros and Cons

Pros

- **Compatibility guarantee**: You can be sure that the products you are getting from a factory are compatible with each other.
- **Avoids tight coupling** between concrete products and client code.
- **Single Responsibility Principle (SRP)**: You can extract the product creation code into one place, making the code easier to support.
- **Open/Closed Principle (OCP)**: You can introduce new variants of products without breaking existing client code.

Cons

- **Increased complexity**: The code may become more complicated than it should be, since a lot of new interfaces and classes are introduced along with the pattern.

Abstract Factory - Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated, more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complex).
- **Builder** focuses on constructing complex objects step by step. **Abstract Factory** specializes in creating families of related objects. Abstract Factory returns the product immediately, whereas Builder lets you run some additional construction steps before fetching the product.
- **Abstract Factory** classes are often based on a set of **Factory Methods**, but you can also use **Prototype** to compose the methods on these classes.
- **Abstract Factory** can serve as an alternative to **Facade** when you only want to hide the way the subsystem objects are created from the client code.
- You can use **Abstract Factory** along with **Bridge**. This pairing is useful when some abstractions defined by Bridge can only work with specific implementations.
- **Abstract Factories**, **Builders**, and **Prototypes** can all be implemented as **Singletons**.

Abstract Factory - Java Code Example

```java
// Abstract Products
interface Chair {
    void sitOn();
    String getStyle();
}

interface Sofa {
    void lieOn();
```

```
    String getStyle();
}

interface CoffeeTable {
    void placeItems();
    String getStyle();
}
```

Abstract Factory - Java Code Example (cont.)

```
// Concrete Products - Modern Family
class ModernChair implements Chair {
    public void sitOn() {
        System.out.println("Sitting on a modern chair.");
    }
    public String getStyle() { return "Modern"; }
}

class ModernSofa implements Sofa {
    public void lieOn() {
        System.out.println("Lying on a modern sofa.");
    }
    public String getStyle() { return "Modern"; }
}

class ModernCoffeeTable implements CoffeeTable {
    public void placeItems() {
        System.out.println("Placing items on modern table.");
    }
    public String getStyle() { return "Modern"; }
}
```

Abstract Factory - Java Code Example (cont.)

```
// Concrete Products - Victorian Family
class VictorianChair implements Chair {
    public void sitOn() {
        System.out.println("Sitting on a Victorian chair.");
    }
    public String getStyle() { return "Victorian"; }
}

class VictorianSofa implements Sofa {
    public void lieOn() {
        System.out.println("Lying on a Victorian sofa.");
    }
    public String getStyle() { return "Victorian"; }
}

class VictorianCoffeeTable implements CoffeeTable {
    public void placeItems() {
        System.out.println("Placing items on Victorian table.");
    }
    public String getStyle() { return "Victorian"; }
```

```
}
```

Abstract Factory - Java Code Example (cont.)

```
// Abstract Factory
interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
    CoffeeTable createCoffeeTable();
}

// Concrete Factories
class ModernFurnitureFactory implements FurnitureFactory {
    public Chair createChair() { return new ModernChair(); }
    public Sofa createSofa() { return new ModernSofa(); }
    public CoffeeTable createCoffeeTable() {
        return new ModernCoffeeTable();
    }
}

class VictorianFurnitureFactory implements FurnitureFactory {
    public Chair createChair() { return new VictorianChair(); }
    public Sofa createSofa() { return new VictorianSofa(); }
    public CoffeeTable createCoffeeTable() {
        return new VictorianCoffeeTable();
    }
}
```

Abstract Factory - Java Code Example (cont.)

```
// Client code
public class AbstractFactoryDemo {
    private Chair chair;
    private Sofa sofa;
    private CoffeeTable table;

    public AbstractFactoryDemo(FurnitureFactory factory) {
        chair = factory.createChair();
        sofa = factory.createSofa();
        table = factory.createCoffeeTable();
    }

    public void describeFurniture() {
        System.out.println("Style: " + chair.getStyle());
        chair.sitOn();
        sofa.lieOn();
        table.placeItems();
    }

    public static void main(String[] args) {
        // Use Modern furniture
        FurnitureFactory factory = new ModernFurnitureFactory();
        AbstractFactoryDemo room = new AbstractFactoryDemo(factory);
        room.describeFurniture();
```

```
        // Output: Style: Modern
        //         Sitting on a modern chair.
        //         Lying on a modern sofa.
        //         Placing items on modern table.
    }
}
```

Module C: Takeaways

- **Abstract Factory** lets you produce families of related objects without specifying their concrete classes.
- It solves the problem of ensuring **product compatibility** within a family (e.g., Modern chair + Modern sofa, never mixing styles).
- The pattern uses **five participants**: Abstract Products, Concrete Products, Abstract Factory, Concrete Factories, and Client.
- It supports **OCP** (add new families without changing existing code) and **SRP** (product creation is centralized).
- Abstract Factory is often implemented using **Factory Methods** internally, and can also use **Prototype** for composition.
- The main trade-off is **increased number of interfaces and classes**.
- Abstract Factories, Builders, and Prototypes can all be implemented as **Singletons**.

Module D: Builder Pattern

Builder Pattern - Overview

- **Category**: Creational Pattern
- **Complexity**: Medium
- **Popularity**: High

Intent: Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.

Reference: RefactoringGuru - Builder[7]

Builder - The Problem (Subclass Explosion)

Imagine a complex object that requires laborious, step-by-step initialization of many fields and nested objects. Such initialization code is usually buried inside a monstrous constructor with lots of parameters. Or even worse: scattered all over the client code.

Example: Building a `House` object.

- A simple house needs walls, a floor, a door, windows, and a roof.
- But what if you want a bigger, brighter house, with a backyard, a swimming pool, a garage, and other goodies?

The simplest solution is to extend the base `House` class and create subclasses for every combination of parameters. But eventually you will end up with a **considerable number of subclasses**. Any new parameter will require growing this hierarchy even more.

---

[7] https://refactoring.guru/design-patterns/builder

Builder - The Problem (Telescoping Constructor)

There is another approach that does not involve breeding subclasses. You can create a giant constructor right in the base `House` class with all possible parameters that control the house object.

```
// Telescoping constructor anti-pattern
class House {
    House(int windows, int doors, int rooms,
          boolean hasGarage, boolean hasSwimPool,
          boolean hasStatues, boolean hasGarden,
          boolean hasYard, boolean hasFence) {
        // ...
    }
}

// Client code - very hard to read
new House(4, 2, 5, true, false, false, true, true, false);
```

In most cases, **most of the parameters will be unused**, making the constructor calls pretty ugly.

Builder - The Solution

The Builder pattern suggests that you extract the object construction code out of its own class and move it to separate objects called **builders**.

The pattern organizes object construction into a set of steps (`buildWalls`, `buildDoor`, etc.). To create an object, you execute a series of these steps on a builder object. The important part is that **you don't need to call all of the steps**. You can call only those steps that are necessary for producing a particular configuration of an object.

Director (optional): You can further extract a series of calls to the builder steps into a separate class called the **director**. The director defines the order in which to execute the building steps, while the builder provides the implementation for those steps.

- Having a director class is not strictly necessary.
- However, the director class might be a good place to put various construction routines so you can reuse them across your program.

Builder - Structure

The Builder pattern has five key participants:

1. **Builder Interface** declares product construction steps that are common to all types of builders.
2. **Concrete Builders** provide different implementations of the construction steps. Concrete builders may produce products that do not follow the common interface.
3. **Products** are resulting objects. Products constructed by different builders do not have to belong to the same class hierarchy or interface.
4. **Director** (optional) defines the order in which to call construction steps, so you can create and reuse specific configurations of products.
5. **Client** must associate one of the builder objects with the director (or use the builder directly). Usually, it is done just once, via parameters of the director's constructor.

Builder - Class Diagram

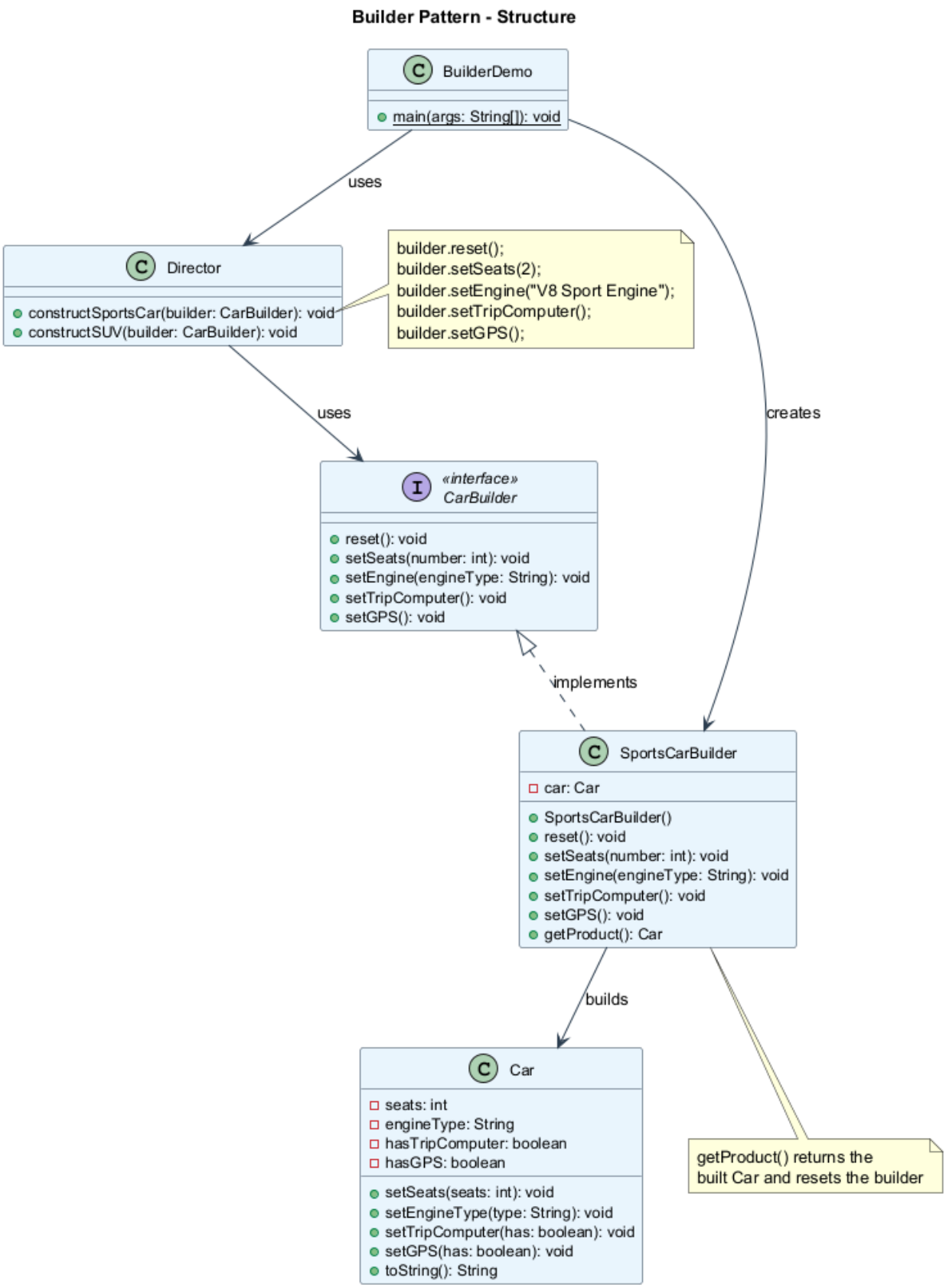


Figure 6: center

Builder - Pseudocode

This example illustrates how the Builder pattern can be used for constructing different types of products (Car and Manual) using the same building steps.

```
// The builder interface specifies methods for creating
// the different parts of the product objects.
interface Builder is
    method reset()
    method setSeats(number)
    method setEngine(engine: Engine)
    method setTripComputer()
    method setGPS()

// Concrete builders implement the builder interface
// and provide specific implementations of building steps.
class CarBuilder implements Builder is
    private field car: Car
    method reset() is
        this.car = new Car()
    method setSeats(number) is
        // Set the number of seats in the car.
    method setEngine(engine: Engine) is
        // Install a given engine.
    method setTripComputer() is
        // Install a trip computer.
    method setGPS() is
        // Install a GPS.
    method getProduct(): Car is
        product = this.car
        this.reset()
        return product
```

Builder - Pseudocode (cont.)

```
// Unlike other creational patterns, Builder lets you
// construct unrelated products with the same process.
class CarManualBuilder implements Builder is
    private field manual: Manual
    method reset() is
        this.manual = new Manual()
    method setSeats(number) is
        // Document car seat features.
    method setEngine(engine: Engine) is
        // Add engine instructions.
    method setTripComputer() is
        // Add trip computer instructions.
    method setGPS() is
        // Add GPS instructions.
    method getProduct(): Manual is
        // Return the manual and reset the builder.

// The director is only responsible for executing the
// building steps in a particular sequence.
class Director is
    method constructSportsCar(builder: Builder) is
        builder.reset()
```

```
        builder.setSeats(2)
        builder.setEngine(new SportEngine())
        builder.setTripComputer()
        builder.setGPS()
    method constructSUV(builder: Builder) is
        builder.reset()
        builder.setSeats(4)
        builder.setEngine(new SUVEngine())
        builder.setGPS()
```

Builder - Pseudocode (cont.)

```
// The client code creates a builder object, passes it
// to the director and then initiates the construction.
class Application is
    method makeCar() is
        director = new Director()

        // Get a car
        CarBuilder builder = new CarBuilder()
        director.constructSportsCar(builder)
        Car car = builder.getProduct()

        // Get a car manual for the same config
        CarManualBuilder manualBuilder = new CarManualBuilder()
        director.constructSportsCar(manualBuilder)
        Manual manual = manualBuilder.getProduct()
        // The final product is retrieved from the builder
        // because the director is not aware of and not
        // dependent on concrete builders and products.
```

Builder - Sequence Diagram

Builder - Applicability

Use the Builder pattern when:

1. **You want to get rid of a "telescoping constructor".**
   - The Builder pattern lets you build objects step by step, using only those steps that you really need. After implementing the pattern, you don't have to cram dozens of parameters into your constructors.
2. **You want your code to be able to create different representations of some product** (e.g., stone and wooden houses).
   - The Builder pattern can be applied when construction of various representations of the product involves similar steps that differ only in the details.
   - The base builder interface defines all possible construction steps, and concrete builders implement these steps to construct particular representations of the product.
3. **You want to construct Composite trees or other complex objects.**
   - The Builder pattern lets you construct products step-by-step. You could defer execution of some steps without breaking the final product. You can even call steps recursively, which comes in handy when you need to build an object tree.

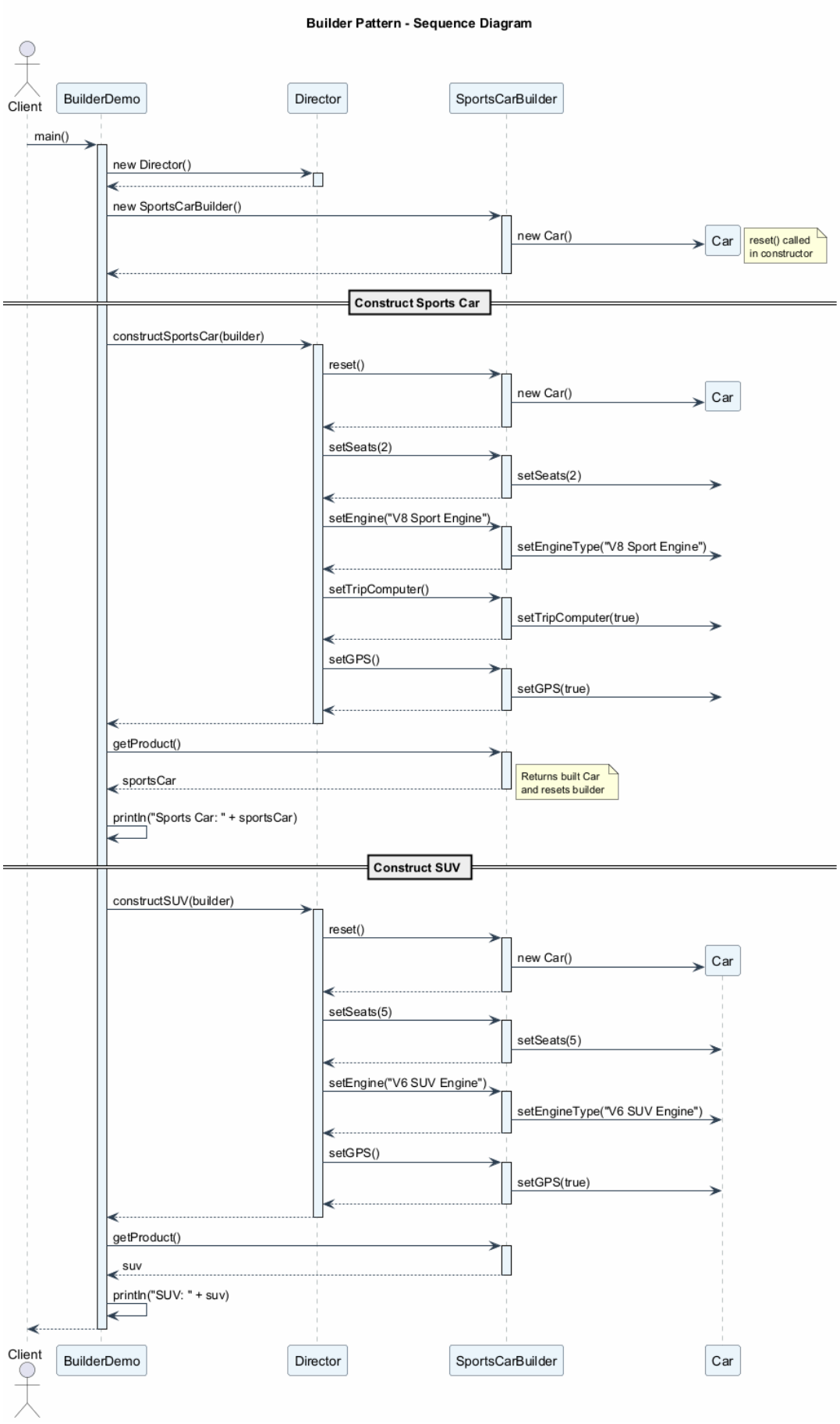


Figure 7: center

Builder - How to Implement

1. **Make sure** that you can clearly define the common construction steps for building all available product representations. Otherwise, you will not be able to proceed with implementing the pattern.
2. **Declare these steps** in the base builder interface.
3. **Create a concrete builder class** for each of the product representations and implement their construction steps.
    - Don't forget about implementing a method for fetching the result of the construction. This method cannot be declared inside the builder interface because various builders may construct products that don't have a common interface.
4. **Think about creating a director class.** It may encapsulate various ways to construct a product using the same builder object.
5. **The client code creates both the builder and the director objects.** Before construction starts, the client must pass a builder object to the director. Usually, the client does this only once, via parameters of the director's constructor or via a setter method.
6. **The construction result can be obtained directly from the builder** only if all products follow the same interface. Otherwise, the client should fetch the result from the builder.

Builder - Pros and Cons

Pros

- **Step-by-step construction**: You can construct objects step-by-step, defer construction steps or run steps recursively.
- **Reusable construction code**: You can reuse the same construction code when building various representations of products.
- **Single Responsibility Principle (SRP)**: You can isolate complex construction code from the business logic of the product.

Cons

- **Increased complexity**: The overall complexity of the code increases since the pattern requires creating multiple new classes.

Builder - Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated, more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complex).
- **Builder** focuses on constructing complex objects step by step. **Abstract Factory** specializes in creating families of related objects. Abstract Factory returns the product immediately, whereas Builder lets you run some additional construction steps before fetching the product.
- You can use **Builder** when creating complex **Composite** trees because you can program its construction steps to work recursively.
- You can combine **Builder** with **Bridge**: the director class plays the role of the abstraction, while different builders act as implementations.
- **Abstract Factories**, **Builders**, and **Prototypes** can all be implemented as **Singletons**.

Builder - Java Code Example

```java
// Product class
class Car {
    private int seats;
    private String engineType;
    private boolean hasTripComputer;
    private boolean hasGPS;

    public void setSeats(int seats) { this.seats = seats; }
    public void setEngineType(String type) {
        this.engineType = type;
    }
    public void setTripComputer(boolean has) {
        this.hasTripComputer = has;
    }
    public void setGPS(boolean has) { this.hasGPS = has; }

    @Override
    public String toString() {
        return "Car{seats=" + seats
            + ", engine='" + engineType + "'"
            + ", tripComputer=" + hasTripComputer
            + ", GPS=" + hasGPS + "}";
    }
}
```

Builder - Java Code Example (cont.)

```java
// Builder interface
interface CarBuilder {
    void reset();
    void setSeats(int number);
    void setEngine(String engineType);
    void setTripComputer();
    void setGPS();
}

// Concrete Builder
class SportsCarBuilder implements CarBuilder {
    private Car car;

    public SportsCarBuilder() { this.reset(); }

    public void reset() { this.car = new Car(); }

    public void setSeats(int number) {
        car.setSeats(number);
    }
    public void setEngine(String engineType) {
        car.setEngineType(engineType);
    }
    public void setTripComputer() {
        car.setTripComputer(true);
    }
    public void setGPS() { car.setGPS(true); }
```

```
    public Car getProduct() {
        Car product = this.car;
        this.reset();
        return product;
    }
}
```

Builder - Java Code Example (cont.)

```
// Director
class Director {
    public void constructSportsCar(CarBuilder builder) {
        builder.reset();
        builder.setSeats(2);
        builder.setEngine("V8 Sport Engine");
        builder.setTripComputer();
        builder.setGPS();
    }

    public void constructSUV(CarBuilder builder) {
        builder.reset();
        builder.setSeats(5);
        builder.setEngine("V6 SUV Engine");
        builder.setGPS();
    }
}

// Client code
public class BuilderDemo {
    public static void main(String[] args) {
        Director director = new Director();
        SportsCarBuilder builder = new SportsCarBuilder();

        director.constructSportsCar(builder);
        Car sportsCar = builder.getProduct();
        System.out.println("Sports Car: " + sportsCar);

        director.constructSUV(builder);
        Car suv = builder.getProduct();
        System.out.println("SUV: " + suv);

        // Expected Output:
        // Sports Car: Car[seats=2, engine=V8 Sport Engine, tripComputer=true, gps=true]
        // SUV: Car[seats=5, engine=V6 SUV Engine, tripComputer=false, gps=true]
    }
}
```

Module D: Takeaways

- **Builder** lets you construct complex objects step by step, using only the steps you need.
- It solves the **telescoping constructor** problem and the **subclass explosion** problem.
- The pattern uses **five participants**: Builder Interface, Concrete Builders, Products, Director, and Client.

- The **Director** is optional but useful for encapsulating common construction routines.
- Builder supports the **Single Responsibility Principle** by isolating construction logic from business logic.
- The main trade-off is **increased code complexity** due to multiple new classes.
- Builder can be combined with **Composite** (for building tree structures), **Bridge**, and can be implemented as a **Singleton**.

Module E: Prototype Pattern

Prototype Pattern - Overview

- **Also Known As**: Clone
- **Category**: Creational Pattern
- **Complexity**: Low
- **Popularity**: Medium

Intent: Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.

Reference: RefactoringGuru - Prototype[8]

Prototype - The Problem

Say you have an object, and you want to create an exact copy of it. How would you do it?

1. **First**, you have to create a new object of the same class.
2. **Then**, you have to go through all the fields of the original object and copy their values over to the new object.

But there are problems:

- **Not all objects can be copied that way** because some of the object's fields may be private and not visible from outside of the object itself.
- **Your code becomes dependent on the class** of the object being copied. You have to know the object's class to create a duplicate, making your code dependent on that class.
- **Sometimes you only know the interface** that the object follows, but not its concrete class. For example, when a method accepts any objects that follow some interface, you don't know the concrete classes of those objects.
- **Complex configurations**: When an object has complex internal state (connections, references, caches), simply knowing its class isn't enough to duplicate it properly.

Prototype - The Solution

The Prototype pattern delegates the cloning process to the actual objects that are being cloned. The pattern declares a common interface for all objects that support cloning. This interface lets you clone an object without coupling your code to the class of that object. Usually, such an interface contains just a single `clone` method.

How it works:

- An object that supports cloning is called a **prototype**.
- When your objects have dozens of fields and hundreds of possible configurations, cloning them might serve as an alternative to subclassing.

[8] https://refactoring.guru/design-patterns/prototype

- You create a set of objects, configured in various ways. When you need an object like the one you have configured, you just clone a prototype instead of constructing a new object from scratch.

Pre-built prototypes: The pattern provides a way to create pre-built prototype objects. You can think of it as an analog of a catalog. These prototypes can have complex state already configured.

Prototype - The Solution (cont.)

Prototype Registry (Cache):

An optional addition to the pattern is a **Prototype Registry** that provides an easy way to access frequently-used prototypes. It stores a set of pre-built objects that are ready to be copied.

The simplest prototype registry is a **name -> prototype** hash map. However, if you need better search criteria than a simple name, you can build a much more robust version of the registry.

```
PrototypeRegistry

  items: Map<String, Prototype>

  addItem(id, prototype)
  getById(id): Prototype
  getByColor(color): Prototype
```

Prototype - Structure

Basic Implementation:

1. **Prototype Interface** declares the cloning methods. In most cases, it is a single `clone` method.
2. **Concrete Prototype** class implements the cloning method. In addition to copying the original object's data to the clone, this method may also handle some edge cases of the cloning process related to cloning linked objects, untangling recursive dependencies, etc.
3. **Client** can produce a copy of any object that follows the prototype interface.

Registry Implementation (variant):

4. **Prototype Registry** provides an easy way to access frequently-used prototypes. It stores a set of pre-built objects that are ready to be copied. The simplest registry is a `name -> prototype` hash map.

Prototype - Class Diagram

Prototype - Pseudocode

In this example, the Prototype pattern lets you produce exact copies of geometric objects, without coupling the code to their classes.

```
// Base prototype
abstract class Shape is
    field X: int
    field Y: int
    field color: string

    // A regular constructor
```

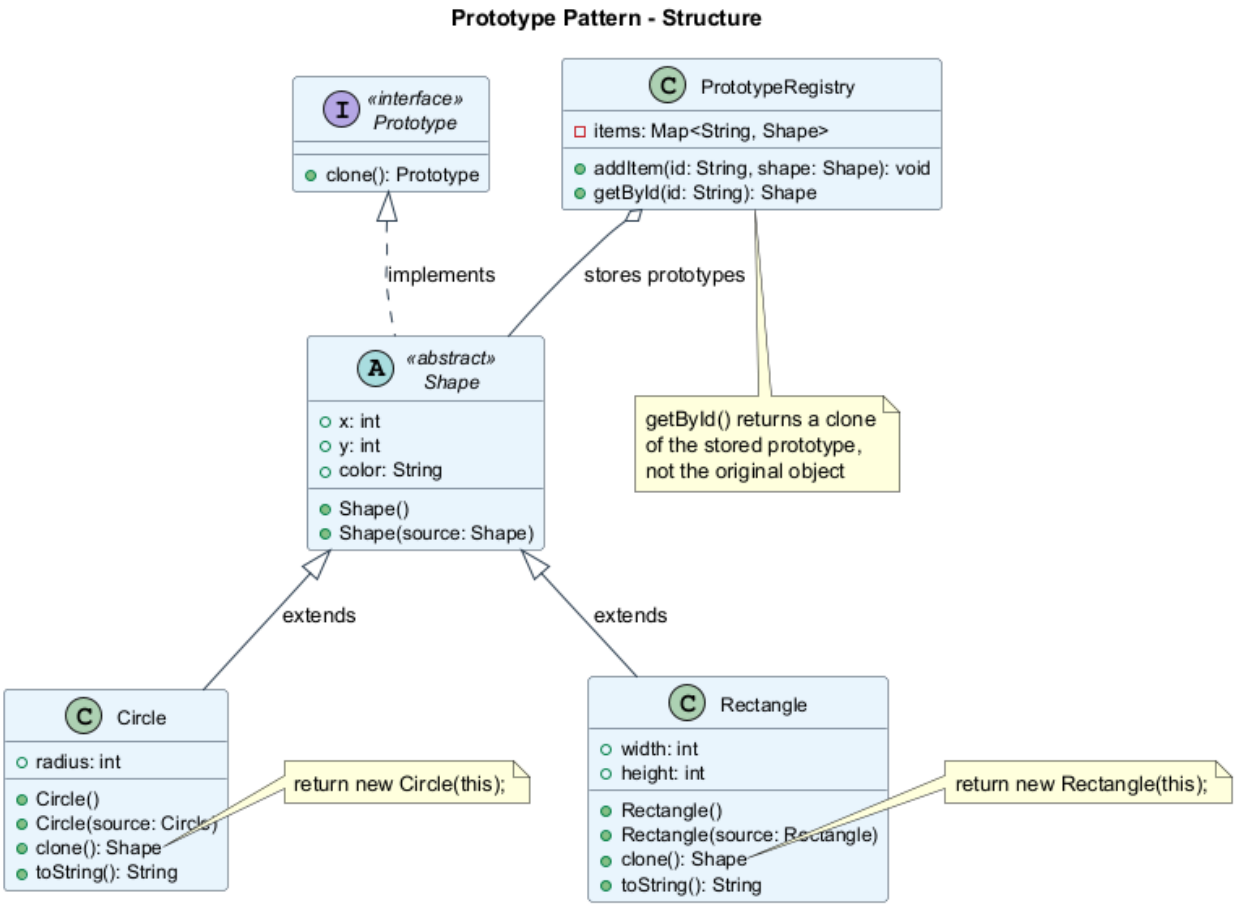


Figure 8: center

```
    constructor Shape() is
        // ...

    // The prototype constructor. A fresh object is
    // initialized with values from the existing object.
    constructor Shape(source: Shape) is
        this()
        this.X = source.X
        this.Y = source.Y
        this.color = source.color

    // The clone operation returns one of the Shape
    // subclasses.
    abstract method clone(): Shape
```

Prototype - Pseudocode (cont.)

```
class Rectangle extends Shape is
    field width: int
    field height: int

    constructor Rectangle(source: Rectangle) is
        super(source)
        this.width = source.width
        this.height = source.height

    method clone(): Shape is
        return new Rectangle(this)

class Circle extends Shape is
    field radius: int

    constructor Circle(source: Circle) is
        super(source)
        this.radius = source.radius

    method clone(): Shape is
        return new Circle(this)
```

Prototype - Pseudocode (cont.)

```
// Somewhere in the client code.
class Application is
    field shapes: array of Shape

    constructor Application() is
        Circle circle = new Circle()
        circle.X = 10
        circle.Y = 10
        circle.radius = 20
        shapes.add(circle)

        Circle anotherCircle = circle.clone()
        shapes.add(anotherCircle)
```

```
    // anotherCircle is an exact copy of circle

    Rectangle rect = new Rectangle()
    rect.width = 10
    rect.height = 20
    shapes.add(rect)

method businessLogic() is
    // Prototype rocks because it lets you produce
    // a copy of an object without knowing anything
    // about its type.
    Array shapesCopy = new Array of Shapes

    foreach (s in shapes) do
        shapesCopy.add(s.clone())
    // shapesCopy contains exact copies of shapes
```

Prototype - Sequence Diagram

Prototype - Applicability

Use the Prototype pattern when:

1. **Your code should not depend on the concrete classes of objects that you need to copy.**
   - This happens a lot when your code works with objects passed to you from 3rd-party code via some interface. The concrete classes of these objects are unknown, and you could not depend on them even if you wanted to.
   - The Prototype pattern provides the client code with a general interface for working with all objects that support cloning. This interface makes the client code independent from the concrete classes of objects that it clones.
2. **You want to reduce the number of subclasses that only differ in the way they initialize their respective objects.**
   - Somebody could have created these subclasses to be able to create objects with a specific configuration.
   - The Prototype pattern lets you use a set of pre-built objects configured in various ways as prototypes. Instead of instantiating a subclass that matches some configuration, the client can simply look for an appropriate prototype and clone it.

Prototype - How to Implement

1. **Create the prototype interface** and declare the `clone` method in it. Or just add the method to all classes of an existing class hierarchy, if you have one.
2. **A prototype class must define the alternative constructor** that accepts an object of that class as an argument. The constructor must copy the values of all fields defined in the class from the passed object into the newly created instance. If you are changing a subclass, you must call the parent constructor to let the superclass handle the cloning of its private fields.
3. **The cloning method usually consists of just one line**: running a `new` operator with the prototypical version of the constructor. Note, every class must explicitly override the cloning method and use its own class name along with the `new` operator. Otherwise, the cloning method may produce an object of a parent class.
4. **Optionally, create a centralized prototype registry** to store a catalog of frequently used prototypes.

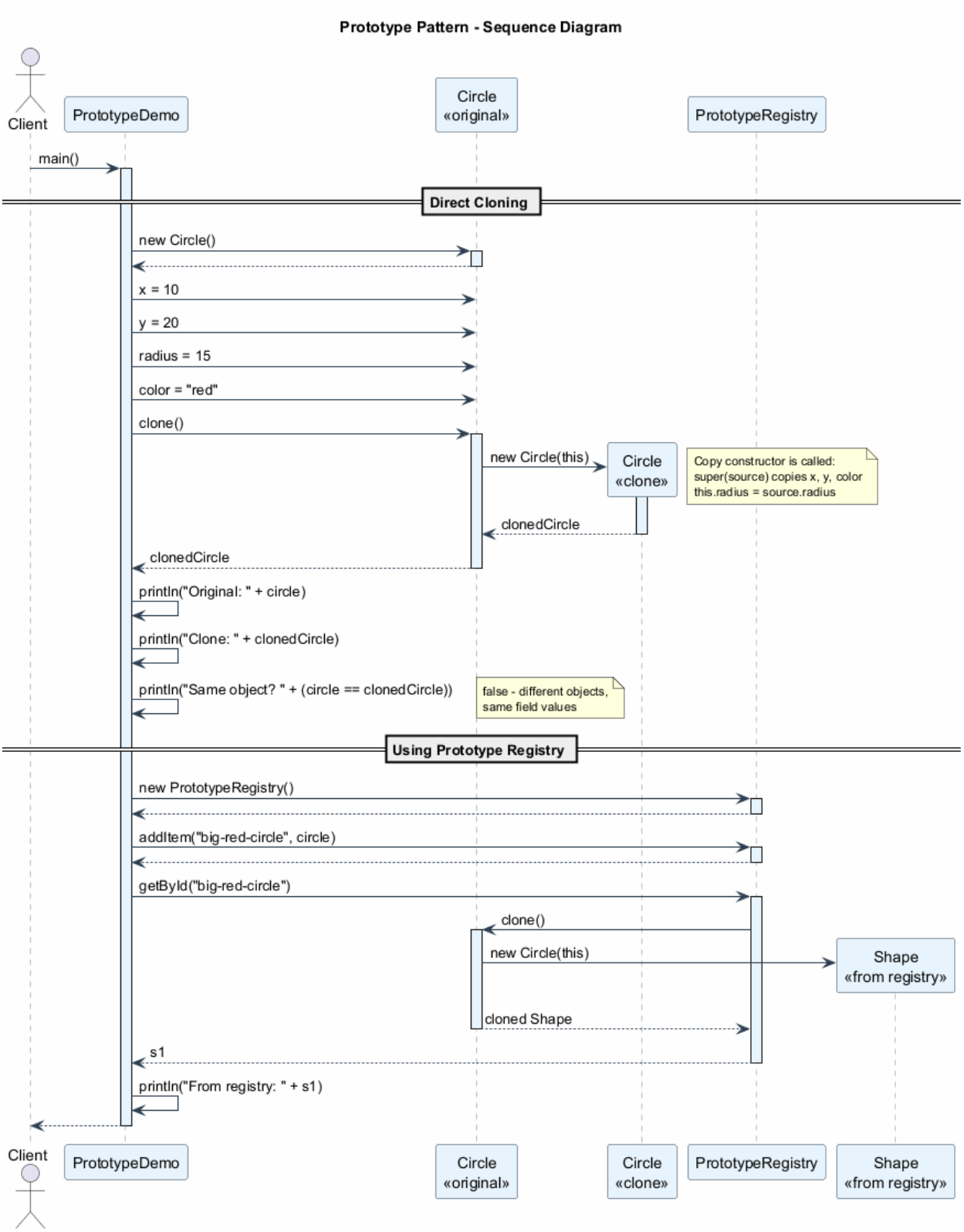


Figure 9: center

5. **You can implement the registry as a new factory class** or put it in the base prototype class with a static method for fetching the prototype. This method should search for a prototype based on search criteria that the client code passes to the method.

Prototype - Pros and Cons

Pros

- **Clone objects without coupling** to their concrete classes.
- **Get rid of repeated initialization code** in favor of cloning pre-built prototypes.
- **Produce complex objects more conveniently**: when objects have complex internal state, cloning is faster and easier than building from scratch.
- **Alternative to inheritance**: You get an alternative for dealing with configuration presets for complex objects. Instead of creating subclasses for each configuration, you can create prototypes with different configurations and clone them.

Cons

- **Cloning complex objects with circular references** might be very tricky. Deep cloning objects that contain references to other objects requires careful handling of circular dependencies.

Prototype - Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated, more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complex).
- **Abstract Factory** classes are often based on a set of **Factory Methods**, but you can also use **Prototype** to compose the methods on these classes.
- **Prototype** can help when you need to save copies of **Commands** into history (Memento pattern alternative).
- Designs that make heavy use of **Composite** and **Decorator** can often benefit from using **Prototype**. Applying the pattern lets you clone complex structures instead of re-constructing them from scratch.
- **Prototype** is not based on inheritance, so it does not have its drawbacks. On the other hand, Prototype requires a complicated initialization of the cloned object. **Factory Method** is based on inheritance but does not require an initialization step.
- Sometimes **Prototype** can be a simpler alternative to **Memento**. This works if the object, the state of which you want to store in the history, is fairly straightforward and does not have links to external resources, or the links are easy to re-establish.
- **Abstract Factories**, **Builders**, and **Prototypes** can all be implemented as **Singletons**.

Prototype - Java Code Example

```java
// Prototype interface
interface Prototype {
    Prototype clone();
}

// Abstract base class
abstract class Shape implements Prototype {
    public int x;
    public int y;
    public String color;
```

```
    public Shape() {}

    // Copy constructor
    public Shape(Shape source) {
        this.x = source.x;
        this.y = source.y;
        this.color = source.color;
    }
}
```

Prototype - Java Code Example (cont.)

```
// Concrete Prototype - Circle
class Circle extends Shape {
    public int radius;

    public Circle() {}

    public Circle(Circle source) {
        super(source);
        this.radius = source.radius;
    }

    @Override
    public Shape clone() {
        return new Circle(this);
    }

    @Override
    public String toString() {
        return "Circle{x=" + x + ", y=" + y
            + ", color='" + color + "'"
            + ", radius=" + radius + "}";
    }
}
```

Prototype - Java Code Example (cont.)

```
// Concrete Prototype - Rectangle
class Rectangle extends Shape {
    public int width;
    public int height;

    public Rectangle() {}

    public Rectangle(Rectangle source) {
        super(source);
        this.width = source.width;
        this.height = source.height;
    }

    @Override
    public Shape clone() {
```

```
        return new Rectangle(this);
    }

    @Override
    public String toString() {
        return "Rectangle{x=" + x + ", y=" + y
            + ", color='" + color + "'"
            + ", width=" + width
            + ", height=" + height + "}";
    }
}
```

Prototype - Java Code Example (cont.)

```
// Prototype Registry
import java.util.HashMap;
import java.util.Map;

class PrototypeRegistry {
    private Map<String, Shape> items = new HashMap<>();

    public void addItem(String id, Shape shape) {
        items.put(id, shape);
    }

    public Shape getById(String id) {
        return (Shape) items.get(id).clone();
    }
}
```

Prototype - Java Code Example (cont.)

```
// Client code
public class PrototypeDemo {
    public static void main(String[] args) {
        // Create original shapes
        Circle circle = new Circle();
        circle.x = 10;
        circle.y = 20;
        circle.radius = 15;
        circle.color = "red";

        // Clone the circle
        Circle clonedCircle = (Circle) circle.clone();
        System.out.println("Original: " + circle);
        System.out.println("Clone:    " + clonedCircle);
        System.out.println("Same object? " +
            (circle == clonedCircle)); // false

        // Use prototype registry
        PrototypeRegistry registry = new PrototypeRegistry();
        registry.addItem("big-red-circle", circle);

        Rectangle rect = new Rectangle();
```

```
        rect.x = 0; rect.y = 0;
        rect.width = 100; rect.height = 50;
        rect.color = "blue";
        registry.addItem("blue-rect", rect);

        // Get clones from registry
        Shape s1 = registry.getById("big-red-circle");
        Shape s2 = registry.getById("blue-rect");
        System.out.println("From registry: " + s1);
        System.out.println("From registry: " + s2);

        // Expected Output:
        // Original: Circle[x=10, y=20, radius=15, color=red]
        // Clone:    Circle[x=10, y=20, radius=15, color=red]
        // Same object? false
        // From registry: Circle[x=10, y=20, radius=15, color=red]
        // From registry: Rectangle[x=0, y=0, width=100, height=50, color=blue]
    }
}
```

Module E: Takeaways

- **Prototype** (also known as **Clone**) lets you copy existing objects without making your code dependent on their classes.
- It solves the problem of **class dependency** when copying objects and handles the issue of **private fields** that cannot be accessed from outside.
- The pattern uses a **clone interface**, **concrete prototypes** with copy constructors, and an optional **Prototype Registry**.
- Prototype is an **alternative to inheritance** for dealing with configuration presets: instead of subclasses, use pre-configured prototypes and clone them.
- The main challenge is handling **circular references** during deep cloning.
- Prototype can be combined with **Composite** and **Decorator** patterns to clone complex structures.
- Unlike **Factory Method**, Prototype is **not based on inheritance** but requires a proper initialization of the cloned object.

Module F: Singleton Pattern

Singleton Pattern - Overview

- **Category**: Creational Pattern
- **Complexity**: Low
- **Popularity**: Very High (also controversial)

Intent: Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

Reference: RefactoringGuru - Singleton[9]

Singleton - The Problem

The Singleton pattern solves two problems at the same time (violating the Single Responsibility Principle):

[9] https://refactoring.guru/design-patterns/singleton

1. **Ensure that a class has just a single instance.**
   - Why would anyone want to control how many instances a class has? The most common reason is to **control access to some shared resource** – for example, a database or a file.
   - It works like this: imagine that you created an object, but after a while decided to create a new one. Instead of receiving a fresh object, you get the one that was already created.
   - Note that this behavior is impossible to implement with a regular constructor since a constructor call must always return a new object by design.
2. **Provide a global access point to that instance.**
   - Remember global variables? While they are very handy, they are also very unsafe because any code can potentially overwrite the contents of those variables and crash the app.
   - Just like a global variable, the Singleton pattern lets you access some object from anywhere in the program. However, it also **protects that instance from being overwritten** by other code.

Singleton - The Solution

All implementations of the Singleton have two common steps:

1. **Make the default constructor private**, to prevent other objects from using the `new` operator with the Singleton class.
2. **Create a static creation method** that acts as a constructor. Under the hood, this method calls the private constructor to create an object and saves it in a static field. All following calls to this method return the cached object.

If your code has access to the Singleton class, then it is able to call the Singleton's static method. So whenever that method is called, the same object is always returned.

```
Singleton

  - instance: Singleton (static)
  - Singleton() (private)

  + getInstance(): Singleton
  + businessLogic()
```

Singleton - Structure

The **Singleton** class declares the static method `getInstance` that returns the same instance of its own class.

- The Singleton's constructor should be hidden from the client code.
- Calling the `getInstance` method should be the only way of getting the Singleton object.

Key structural elements:

Element	Description
`- instance: Singleton`	Static field holding the single instance
`- Singleton()`	Private constructor preventing external instantiation
`+ getInstance()`	Public static method returning the single instance
Business methods	Any methods the Singleton class needs to provide

---

Singleton - Class Diagram

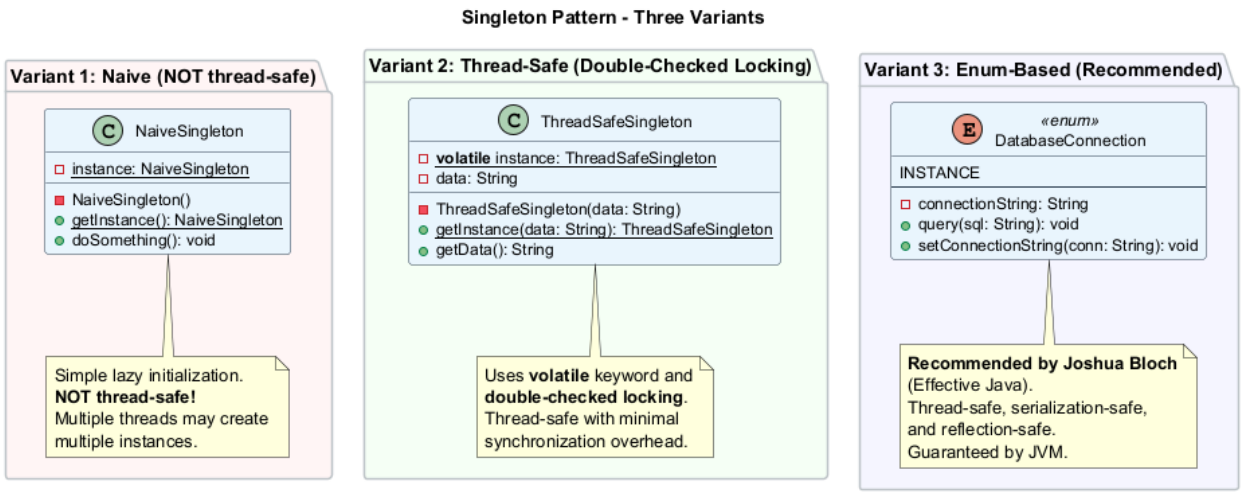


Figure 10: center

Singleton - Pseudocode

In this example, the database connection class acts as a Singleton. This class does not have a public constructor, so the only way to get its object is to call the `getInstance` method. This method caches the first created object and returns it in all subsequent calls.

```
class Database is
    // The field for storing the singleton instance
    // should be declared static.
    private static field instance: Database

    // The singleton's constructor should always be
    // private to prevent direct construction calls
    // with the `new` operator.
    private constructor Database() is
        // Some initialization code, such as the actual
        // connection to a database server.

    // The static method that controls the access to
    // the singleton instance.
    public static method getInstance() is
        if (Database.instance == null) then
            acquireThreadLock() and then
                // Ensure that the instance has not yet been
                // initialized by another thread.
                if (Database.instance == null) then
                    Database.instance = new Database()
        return Database.instance

    public method query(sql) is
        // All database queries go through this method.
```

Singleton - Sequence Diagram

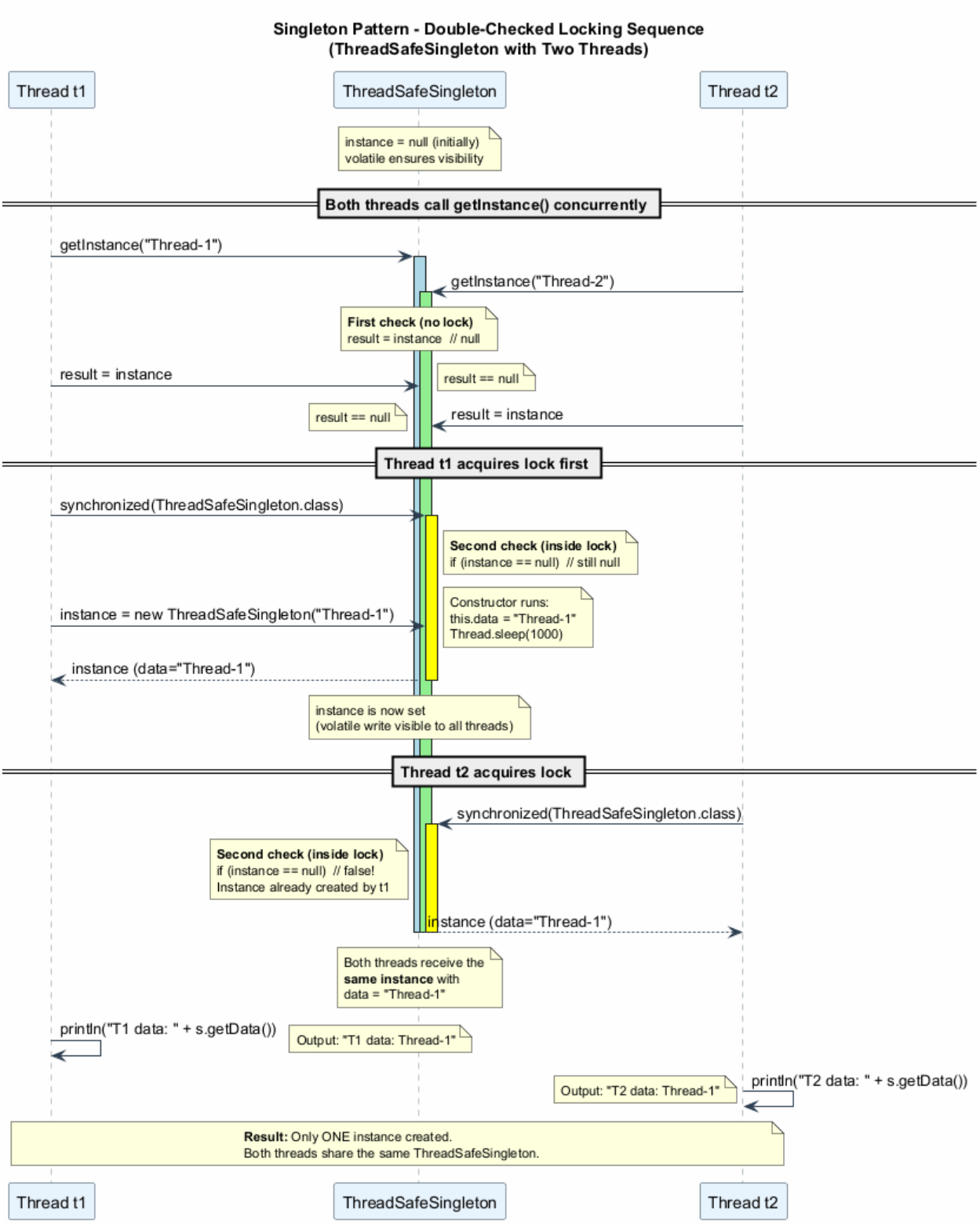


Figure 11: center

Singleton - Thread Safety Issue

Naive implementation is NOT thread-safe:

```
// BROKEN: Race condition possible
public static Singleton getInstance() {
    if (instance == null) {          // Thread A checks
        // Thread B also checks - both see null!
        instance = new Singleton();  // Both create instances!
    }
    return instance;
}
```

Timeline of the race condition:

Time	Thread A	Thread B
T1	Checks `instance == null` (true)	–
T2	–	Checks `instance == null` (true)
T3	Creates new instance	–
T4	–	Creates ANOTHER new instance
T5	Two different instances exist!	Singleton violated!

Singleton - Applicability

Use the Singleton pattern when:

1. **A class in your program should have just a single instance** available to all clients; for example, a single database object shared by different parts of the program.
   - The Singleton pattern disables all other means of creating objects of a class except for the special creation method. This method either creates a new object or returns an existing one if it has already been created.
2. **You need stricter control over global variables.**
   - Unlike global variables, the Singleton pattern guarantees that there is just one instance of a class. Nothing, except for the Singleton class itself, can replace the cached instance.
   - Note that you can always adjust this limitation and allow creating any number of Singleton instances. The only piece of code that needs changing is the body of the `getInstance` method.

Singleton - How to Implement

1. **Add a private static field** to the class for storing the singleton instance.
2. **Declare a public static creation method** for getting the singleton instance.
3. **Implement "lazy initialization"** inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
4. **Make the constructor of the class private.** The static method of the class will still be able to call the constructor, but not the other objects.
5. **Go over the client code** and replace all direct calls to the singleton's constructor with calls to its static creation method.

Singleton - Pros and Cons

Pros

- **Single instance guarantee**: You can be sure that a class has only a single instance.
- **Global access point**: You gain a global access point to that instance.
- **Lazy initialization**: The singleton object is initialized only when it is requested for the first time, saving resources if it is never used.

Cons

- **Violates the Single Responsibility Principle (SRP)**: The pattern solves two problems at the same time (single instance + global access).
- **Can mask bad design**: For instance, when the components of the program know too much about each other.
- **Requires special treatment in a multithreaded environment**: so that multiple threads will not create a singleton object several times.
- **Difficult to unit test**: The Singleton's constructor is private and overriding static methods is impossible in most languages. You need to think of a creative way to mock the singleton. Or just don't write the tests. Or don't use the Singleton pattern.

Singleton - Relations with Other Patterns

- A **Facade** class can often be transformed into a **Singleton** since a single facade object is sufficient in most cases.
- **Flyweight** would resemble **Singleton** if you somehow managed to reduce all shared states of the objects to just one flyweight object. But there are two fundamental differences:
  1. There should be only one Singleton instance, whereas a Flyweight class can have multiple instances with different intrinsic states.
  2. The Singleton object can be mutable. Flyweight objects are immutable.
- **Abstract Factories**, **Builders**, and **Prototypes** can all be implemented as **Singletons**.

Singleton - Java Code Example (Naive)

```java
// Simple but NOT thread-safe Singleton
public class NaiveSingleton {
    private static NaiveSingleton instance;

    // Private constructor
    private NaiveSingleton() {
        // Initialization code
        System.out.println("Singleton instance created.");
    }

    // Static accessor method
    public static NaiveSingleton getInstance() {
        if (instance == null) {
            instance = new NaiveSingleton();
        }
        return instance;
    }

    public void doSomething() {
        System.out.println("Singleton is working.");
```

```
    }
}
```

Warning: This implementation is NOT thread-safe. Multiple threads may create multiple instances.

Singleton - Java Code Example (Thread-Safe)

```java
// Thread-safe Singleton with double-checked locking
public class ThreadSafeSingleton {
    // volatile ensures visibility across threads
    private static volatile ThreadSafeSingleton instance;

    private String data;

    private ThreadSafeSingleton(String data) {
        this.data = data;
        // Simulate slow initialization
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    // Double-checked locking
    public static ThreadSafeSingleton getInstance(String data) {
        ThreadSafeSingleton result = instance;
        if (result != null) {
            return result;  // Fast path: no locking needed
        }
        synchronized (ThreadSafeSingleton.class) {
            if (instance == null) {
                instance = new ThreadSafeSingleton(data);
            }
            return instance;
        }
    }

    public String getData() { return data; }
}
```

Singleton - Java Code Example (Enum-Based)

```java
// The simplest and most effective Singleton in Java
// Joshua Bloch recommends this approach in Effective Java
public enum DatabaseConnection {
    INSTANCE;

    private String connectionString;

    DatabaseConnection() {
        // Initialization happens once, guaranteed by JVM
        this.connectionString = "jdbc:mysql://localhost:3306/mydb";
        System.out.println("Database connection initialized.");
    }
```

```
    public void query(String sql) {
        System.out.println("Executing query: " + sql
            + " on " + connectionString);
    }

    public void setConnectionString(String conn) {
        this.connectionString = conn;
    }
}

// Usage
class EnumSingletonDemo {
    public static void main(String[] args) {
        DatabaseConnection db = DatabaseConnection.INSTANCE;
        db.query("SELECT * FROM users");

        // Same instance everywhere
        DatabaseConnection db2 = DatabaseConnection.INSTANCE;
        System.out.println("Same? " + (db == db2)); // true
    }
}
```

Singleton - Java Code Example (Complete Demo)

```
// Comprehensive Singleton demonstration
public class SingletonDemo {
    public static void main(String[] args) {
        // Test thread-safe singleton
        System.out.println("--- Thread-Safe Singleton ---");

        Thread t1 = new Thread(() -> {
            ThreadSafeSingleton s =
                ThreadSafeSingleton.getInstance("Thread-1");
            System.out.println("T1 data: " + s.getData());
        });

        Thread t2 = new Thread(() -> {
            ThreadSafeSingleton s =
                ThreadSafeSingleton.getInstance("Thread-2");
            System.out.println("T2 data: " + s.getData());
        });

        t1.start();
        t2.start();

        // Both threads get the SAME instance
        // Only one of "Thread-1" or "Thread-2" will be
        // the data value, depending on which thread
        // created the instance first.

        System.out.println("\n--- Enum Singleton ---");
        DatabaseConnection.INSTANCE.query("SELECT 1");

        // Expected Output (thread order may vary):
        // --- Thread-Safe Singleton ---
```

```
        // T1 data: Thread-1
        // T2 data: Thread-1
        //
        // --- Enum Singleton ---
        // Executing query: SELECT 1 on jdbc:default
    }
}
```

Singleton - Comparison of Implementations

Approach	Thread Safe	Lazy Init	Serialization Safe	Reflection Safe
Naive	No	Yes	No	No
Synchronized method	Yes	Yes	No	No
Double-checked locking	Yes	Yes	No	No
Bill Pugh (Inner class)	Yes	Yes	No	No
Enum	**Yes**	**No**	**Yes**	**Yes**

- The **Enum-based** approach is the recommended way in Java (Effective Java by Joshua Bloch).
- It provides serialization safety (JVM handles it) and reflection safety (cannot create enum instances via reflection).
- The only drawback is that it does not support lazy initialization (enum instances are created when the enum class is loaded).

Module F: Takeaways

- **Singleton** ensures a class has only one instance and provides a global access point to it.
- It solves **two problems**: controlling the number of instances and providing global access.
- The pattern requires a **private constructor** and a **static getInstance() method**.
- **Thread safety** is a critical concern; naive implementations can break under concurrent access.
- Java offers multiple Singleton implementations: naive, synchronized, double-checked locking, inner class (Bill Pugh), and **enum** (recommended).
- Singleton is **controversial**: it violates SRP, can mask bad design, and makes testing difficult.
- Abstract Factories, Builders, and Prototypes can all be implemented as **Singletons**.
- The **Enum-based Singleton** is the gold standard in Java: thread-safe, serialization-safe, and reflection-safe.

References

References

1. RefactoringGuru - Design Patterns[10] - Comprehensive guide to design patterns with examples in multiple languages.

[10] https://refactoring.guru/design-patterns

2. RefactoringGuru - Factory Method[11] - Factory Method pattern: intent, structure, pseudocode, applicability, and implementation.
3. RefactoringGuru - Abstract Factory[12] - Abstract Factory pattern: producing families of related objects.
4. RefactoringGuru - Builder[13] - Builder pattern: constructing complex objects step by step.
5. RefactoringGuru - Prototype[14] - Prototype pattern: copying existing objects without class dependency.
6. RefactoringGuru - Singleton[15] - Singleton pattern: ensuring single instance with global access.
7. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
8. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley. - Recommended Singleton implementation using Enum.

$End - Of - Week - 9 - Module$

[11] https://refactoring.guru/design-patterns/factory-method
[12] https://refactoring.guru/design-patterns/abstract-factory
[13] https://refactoring.guru/design-patterns/builder
[14] https://refactoring.guru/design-patterns/prototype
[15] https://refactoring.guru/design-patterns/singleton